

中山大学计算机学院本科生实验报告

(2024 学年第 1 学期)

课程名称: Data structures and algorithms

任课教师: 张子臻

年级	23 级	专业 (方向)	计算机科学与技术
学号	23320093	姓名	林宏宇
电话	13421145433	Email	linhy69@mail2.sysu.deu.cn
开始日期	2024.11.8	完成日期	2024.11.8

第一题

1. 实验题目

1. 二叉搜索树的遍历

2. 实验目的

1. 给定一组无序整数, 以第一个元素为根节点, 按顺序插入结点生成一棵二叉搜索树, 并对其进行中序遍历和先序遍历。

3. 算法设计

1. 创建二叉树需要的结构体
2. 输入
3. 核心算法: 将结点插入到二叉树进去, 大于根节点的插入到右边, 小于根节点的插入到左边
4. 前序遍历打印结点
5. 中序遍历利用特点 直接输出递增的序列

4. 程序运行与测试

1. 核心函数 插入结点到 BST

// 插入节点到二叉搜索树

```
Node* insert(Node* root, int num) {  
    if (root == NULL) {  
        root = new Node(); // 为新节点分配内存
```

```

    root->data = num;
    root->left = root->right = NULL; // 初始化左右子树为空
    return root;
}
if (num > root->data) {
    root->right = insert(root->right, num); // 插入到右子树
} else {
    root->left = insert(root->left, num); // 插入到左子树
}
return root;
}

```

2. 前序遍历打印结点

// 前序遍历: 打印结点 -> 遍历左子树 -> 遍历右子树

```

void traversal_pre(Node* root) {
    if (root == NULL)
        return;

    cout << root->data << " "; // 打印当前节点
    traversal_pre(root->left); // 遍历左子树
    traversal_pre(root->right); // 遍历右子树
}

```

5. 实验总结与心得

1. 实验题目主要是能掌握前中后序遍历算法
2. 明白二叉搜索树的原理
3. 在分配空间的知识点有点陌生 new 函数

第二题

1. 实验题目

1. Play with Tree

2. 实验目的

1. Given a binary tree, your task is to get the height and size(number of nodes) of the tree.
The Node is defined as follows.

```

struct Node {

Node *lc, *rc;char data;

};

```

3. 算法设计

1. 递归函数，将大问题拆解
2. 每次返回子树的高度和结点个数

4. 程序运行与测试

```

1.void query(const Node *root, int &size, int &height)
{
    if(root==NULL)//该结点空
    {
        size=0;
        height=0;//空结点高度为0
        return;
    }
    int s_left=0;
    int s_right=0;
    int h_left=0;
    int h_right=0;
    query(root->lc,s_left,h_left);
    query(root->rc,s_right,h_right);
    size=s_left+s_right+1;
    height=max(h_left,h_right)+1;//size 和 height 的操作放在后面
}

```

5. 实验总结与心得

1. 结点总数的判断是 $size_left + size_right + 1$
2. 高度的判断是 $\max(height_left, height_right) + 1$
3. 此操作需要放在递归函数的结尾进行

第三题

1. 实验题目

1. Given the pre-order traversal sequence $A[]$ and in-order traversal sequence $B[]$ of a tree, find the post-order traversal sequence.

2. 实验目的

1. Input contains 3 lines;

The first line contains an integer n ($n \leq 100000$), the number of node in the tree.

The second line contains n distinct intergers, which is the sequence $A[]$ ($0 \leq a_0, a_2, \dots, a_{n-1} \leq n-1$).

The third line contains n distinct intergers, which is the sequence $B[]$ ($0 \leq b_0, b_2, \dots, b_{n-1} \leq n-1$).

3. 算法设计

1. 输入

2. 创建结点的结构体

3. 递归函数：利用前序遍历找到根节点，再从中序的数组里将根节点左右序列拆分成左子树部分和右子树部分，再利用左右子树的长度回到前序的数组里，将前序的数组拆分成左子树部分和右子树部分，然后重新对拆分好的左子树的前序和中序序列进行递归。函数的终止条件为序列的长度为 0

4. 输出 后序遍历

4. 程序运行与测试

1. 核心函数部分

```
TreeNode* traversal(vector<int> inorder, vector<int> preorder)
{
    if (preorder.size() == 0)
        return nullptr;

    int num = preorder[0]; // 先序遍历第一个为根节点
    TreeNode* root = new TreeNode(num);

    if (preorder.size() == 1)
    {
        root->left = NULL;
```

```

        root->right = NULL;
        return root;
    }

    int index; //在中序中找到根节点
    for (index = 0; index < inorder.size(); index++)
    {
        if (inorder[index] == num)
            break;
    }
    vector<int> leftin(inorder.begin(), inorder.begin() + index);
    vector<int> rightin(inorder.begin() + index + 1, inorder.end()); //切割 inorder
    preorder.erase(preorder.begin()); //删掉刚才创建的结点
    vector<int> leftpre(preorder.begin(), preorder.begin() + leftin.size()); //根据切割下来的 leftin 决定如何切割 postorder
    vector<int> rightpre(preorder.begin() + leftin.size(), preorder.end());

    root->left = traversal(leftin, leftpre);
    root->right = traversal(rightin, rightpre);
    return root;
}

```

5. 实验总结与心得

1. 题目难度较大，需要解决的部分较多，重点在于遍历的递归函数
2. 判断好递归的终止条件，序列的拆分方式，以及对原来的序列的处理
3. 主要逻辑：利用前序遍历找到根节点，再从中序的数组里将根节点左右序列拆分成左子树部分和右子树部分，再利用左右子树的长度回到前序的数组里，将前序的数组拆分成左子树部分和右子树部分，然后重新对拆分好的左子树的前序和中序序列进行递归。函数的终止条件为序列的长度为 0